

Loom Walkthrough Script: Android 3D Multi-Model Viewer

Natural intermediate English spoken script with screen cues, anti-collision label layout, and technical highlights.

 Target: 5–6 Minutes

 Tone: Natural Intermediate English

 Stack: Kotlin • Jetpack Compose • SceneView (Filament)

 5 Models: Airplane • Camera • Lantern • Mars Rover • Damaged Helmet

 Features: Zero-Overlap Labels • Surface Z-Order • Dual Gestures

1. Introduction

30–45 sec

What to show on screen

Webcam and screen side-by-side with Android Studio on the left and the app running on your device/emulator on the right.

WHAT TO SAY

"Hi everyone! In this video, I'm excited to show you our Android 3D Multi-Model Viewer app.

The goal of this project was to build a single-activity Android app where users can open, move around, and interact with multiple 3D models at the same time on a single 2D canvas—while keeping everything super smooth, even on budget devices.

We bundled 5 diverse 3D models in GLB format: **the Damaged Helmet, the Airplane, the Antique Camera, the Lantern, and the Mars Rover.**

We built it with independent container drag and pinch-resize gestures, a dedicated 3D interaction mode to rotate and zoom models, and real-time 2D labels with smart connector lines that track 3D parts with zero overlapping.

Under the hood, we used **Kotlin, Jetpack Compose** for our UI, and **SceneView** powered by Google's **Filament** 3D engine, supporting Android 7.0 all the way up to Android 15.

Let's jump right into the live app demo!"

2. App Demonstration

2-3 min

What to show on screen

Bring your phone or emulator front and center. Follow each step as you speak.

Step 2.1: Launch & Add First Model

ACTION & SPEECH

(Action: Tap "+ Add Model", tap the Damaged Helmet)

"When we open the app, we see a clean, dark canvas with an '+ Add Model' button at the bottom. Let's tap it. This opens our model selector bottom sheet with thumbnail previews. Let's pick our first model—the **Damaged Helmet**. Notice that as soon as it loads, the model is automatically centered inside its container box at (0, 0, 0)."

Step 2.2: Add Multiple Models (5 Models)

ACTION & SPEECH

(Action: Add Airplane, Antique Camera, Lantern, and Mars Rover)

"Now let's add the other four models: **the Airplane, the Camera, the Lantern, and the Mars Rover**. Look at that: we now have all 5 different 3D models actively rendering on the screen at the same time, each with full PBR textures, lighting, and real-time shadows."

Step 2.3: Drag & Resize Containers

ACTION & SPEECH

(Action: Drag a container with 1 finger; overlap another container; 2-finger pinch to resize)

"In **Normal Mode**, each container works just like an interactive card. With one finger, I can drag any container anywhere on the screen. When I touch or drag a container, it automatically comes to the front. Even when two containers overlap, the 3D model stays on top of the container underneath without any weird clipping. And if I use two fingers, I can pinch to resize the container smoothly. The movement is natural and doesn't jump across the screen."

Step 2.4: Interaction Mode (Rotate & Zoom)

ACTION & SPEECH

(Action: Tap hand icon on Rover or Camera container; rotate and pinch-zoom)

"Now, what if we want to inspect a model closely? We just tap this hand icon in the top-right corner of its container. This turns on **Interaction Mode**. Now the container stays locked in place, and dragging spins the 3D camera around the model. I can rotate it to see any angle, and pinch to zoom in on the small details. Our mode separation makes sure that rotating the 3D model never accidentally moves the container."

Step 2.5: Part Labels & Anti-Collision Layout

ACTION & SPEECH

(Action: Tap label toggle icon on the container; rotate the model to show badges staying separated)

"Next, let's tap the label button. Immediately, we see clear text badges with a cyan dot and connector lines pointing directly to specific parts of the model. Notice our **anti-collision layout**: even when I rotate the model and parts line up vertically, the badges never overlap or hide each other. Each badge gets its own dedicated vertical lane with clean elbow connector lines. And as I rotate or zoom the 3D model, the labels track their parts smoothly in real time."

Step 2.6: Remove Model

ACTION & SPEECH

(Action: Tap 'X' button on a container)

"When we're done with a model, we tap the 'X' close button. That removes the container and instantly cleans up all its GPU memory."

3. Project Structure

2-3 min

What to show on screen

Switch to Android Studio. Expand project packages: `data` , `ui` , and `assets/models` .

WHAT TO SAY

"Now let's look at how the project is organized in Android Studio. We used clean MVVM architecture with clear layers:

1. `data` package:

- `data/model` : Contains our data classes like `ModelContainerState` , which stores container position, dimensions, z-index, and label state.
- `data/parser/GlbMetadataParser` : This is our custom binary parser. It reads the GLB file directly, finds node positions, and extracts label names from the GLB extras without needing any external files.
- `data/model/ModelAsset` : Provides the catalog for our 5 bundled 3D models stored in `assets/models/` —our Airplane, Camera, Lantern, Mars Rover, and Damaged Helmet.

2. `ui` package:

- `ui/canvas/CanvasViewModel` : Manages our active containers using Kotlin `StateFlow` . It handles adding models, closing them, moving them, and updating their layers.
- `ui/components/DraggableContainer` : Handles container gestures—1-finger dragging and 2-finger pinch resizing.
- `ui/components/SceneViewContainer` : Connects Jetpack Compose to the Filament 3D engine using `AndroidView` . It manages the camera, lighting, and frame-by-frame label projection.
- `ui/components/PartLabelOverlay` : A Compose canvas that solves label collisions and draws the pins, elbow lines, and text badges on top of the 3D view.

This structure keeps our 3D rendering code completely separate from our UI logic, which makes the whole app clean and easy to maintain."

4. Core Implementation

3-5 min

What to show on screen

Open `SceneViewContainer.kt` , `DraggableContainer.kt` , `PartLabelOverlay.kt` , and `GlbMetadataParser.kt` .

4.1 3D Rendering & Auto-Centering

WHAT TO SAY

"First, when loading a 3D model in `SceneViewContainer.kt` , different 3D models often have their pivot points off-center. So when the model loads, we calculate its bounding box center and shift its position by `Float3(-cx, -cy, -cz)` . This guarantees that every single model is perfectly centered at (0, 0, 0) when it appears."

4.2 Smooth Gesture Handling

WHAT TO SAY

"Second, in `DraggableContainer.kt` , handling 1-finger drag and 2-finger pinch without conflicts can be tricky. If you're not careful, putting down a second finger can make the container jump across the screen. To fix that, we wrote a custom gesture pass using Compose's `awaitEachGesture` . When only one finger touches, we treat it as a drag. The exact moment a second finger touches, we stop dragging completely and only calculate the pinch distance for resizing. This makes the gesture feel totally natural."

4.3 Fixing Overlapping 3D Models (Z-Ordering)

WHAT TO SAY

"Third, we solved a really tricky issue with overlapping containers. Under the hood, `SceneView` uses Android `SurfaceView` . In Jetpack Compose, simply setting `Modifier.zIndex()` reorders the Compose card, but the actual 3D surfaces can still draw in the wrong order in Android's hardware compositor. That made 3D models disappear behind other containers.

We fixed this deeply in `updateSurfaceZOrder()` :

1. We bring the view to front in the parent Android view hierarchy.
 2. We dynamically update the `SurfaceView` sub-layer using `setZOrderMediaOverlay(isTop)` .
- Because of this, whenever you tap or drag any container, its 3D model always renders right on top."

4.4 How Labels Are Set & Anti-Collision Layout

WHAT TO SAY

"Fourth, how are labels defined and displayed without overlapping?

There are two ways to set labels in this project:

1. **In 3D Tools like Blender:** You add a Custom Property named `prop` to any mesh or empty node, export as GLB with custom properties enabled, and our `GlbMetadataParser` reads it automatically.
2. **In Kotlin Code:** If a model doesn't have custom properties, `GlbMetadataParser` provides built-in fallback anchors for each model with realistic part positions.

To project these 3D points to 2D screen coordinates, we use camera matrices:

```
clip = cullingProjectionTransform * viewTransform * [x, y, z, 1]
```

And in `PartLabelOverlay.kt` , we built a smart **Anti-Collision Layout Solver**:

- It partitions labels into Left and Right columns.
- It runs relaxation passes that enforce a minimum vertical gap of 8dp between badges.

- It draws two-segment elbow connector lines from the 3D pin to the badge border.
Because of this mathematical solver, **labels never overlap**, even when rotating or viewing from extreme angles."

4.5 Resource Cleanup

WHAT TO SAY

"And fifth, when a container is closed, Compose disposes the view and calls `sceneView.destroy()`, releasing all textures, meshes, and GPU memory right away."

5. Performance

1–2 min

What to show on screen

Show all 5 models running on screen. Spin one model and drag another container at the same time.

WHAT TO SAY

"Running 5 full 3D models at the same time on mobile can be heavy on memory and the GPU. Here's what we did to optimize performance:

- First, by keeping label projection state local inside `SceneViewContainer`, only the 2D overlay redraws on each frame—not the entire screen or `ViewModel`.
- Second, in our container list, we keyed every container by its unique ID. Moving one container never forces other 3D models to reload.
- Third, we set up transparent clearing on Filament's renderer to skip unnecessary draw passes.

During testing on mid-range devices with 3 GB of RAM, memory usage stayed under 250 MB, and the frame rate stayed between 45 and 60 FPS even with all 5 models running at once."

6. Architecture & Design Decisions

1–2 min

What to show on screen

Briefly show `build.gradle.kts` and the clean package layout.

WHAT TO SAY

"A couple quick thoughts on why we made certain tech choices:

- **Why SceneView and Filament?** Writing our own raw OpenGL shaders would mean hundreds of lines of code just to handle basic lighting, shadows, and glTF parsing. SceneView gives us Google's Filament PBR engine with realistic materials and camera controls out of the box.
- **Why Jetpack Compose?** Compose makes managing multiple dynamic containers really straightforward. Adding, removing, or reordering cards is simply driven by our UI state.
- **Trade-offs:** Mixing native `SurfaceView` with Compose requires handling hardware Z-ordering carefully, but the rendering speed of `SurfaceView` is way faster than `TextureView` for multi-model 3D scenes."

7. Conclusion

30 sec

What to show on screen

Full-screen view of your webcam with the app running in your hands or on the side.

WHAT TO SAY

"To wrap things up:

What works:

- All 5 models—the Airplane, Camera, Lantern, Mars Rover, and Damaged Helmet—load, center, and render smoothly.
- Dragging and resizing work without any jumping.
- Interaction mode and Normal mode are cleanly separated.
- And 2D part labels stay clearly visible with zero overlapping thanks to our anti-collision layout.

What I'd improve with more time: I'd love to add container snapping so cards don't overlap, and raycast occlusion so labels hide when a part is behind the model.

The code is clean, and the signed release APK is ready in the `release/` folder.

Thanks so much for watching, and have a great day!"

Handy Recording Tips

- **Developer Options:** Turn on "*Show taps*" in Android Developer Settings so the audience can see your finger gestures clearly.
- **Preload Models:** Open the app with 2 models already on screen so you can jump straight into the demo.
- **Natural Pacing:** Pause for 1 second after tapping buttons to let the visual action register before speaking about it.